

QA Agent — How the Machine Works

A from-scratch walkthrough of every stage, traced through one real example

Project: autonomous QA testing agent • Example: the hospital waitlist form

The whole system in one sentence:

Open a browser → **read** the page into a list of structured **Element** objects → **understand** what kind of app it is → **invent** realistic data for the fields → **type** that data into the right widgets → **click** submit (safely) → **watch** for anything that broke.

Seven stages. Each is essentially one file. This document follows the exact order things happen when you run `show_execute` — like following a single drop of water through the whole plumbing.

Our running example: the waitlist page

When you ran the agent against `localhost:5173/opd/waitlist`, the form had these fields. Keep this table in your head — everything below is about turning this picture into actions.

Field on screen	Widget kind	Real options (if any)
First Name	text input	—
Last Name	text input	—
Age	text input	—
Phone	text input	—
Doctor	MUI Select (fake dropdown)	Dr. Vivek BHARAMBE, Dr. Sujata BHARAMBE, Other
Visit Reason	MUI Select	ANC, Infertility, Bleeding, Gynaec, Others
Medical Record	text input	—
Add to Waiting List	submit button	—

STAGE 0

Boot up

`show_execute.py`

```

llm = get_provider()                # 1. pick the AI brain
async with BrowserSession(headless=False): # 2. open a real Chrome window
    observer = Observer(session.page)    # 3. start watching for errors
    await session.goto(url)              # 4. navigate to the waitlist page

```

get_provider() reads the `LLM_PROVIDER` env var, sees "groq" (the default), and hands back a `GroqProvider`. This is the object that talks to the Groq API whenever any part of the code needs the AI to think. One brain, shared by everyone.

BrowserSession(headless=False) is a context manager. Entering it starts Playwright, launches a visible Chromium window, creates a fresh cookie-less context with a 1440×860 viewport, and opens a blank page. When the `async with` block ends, everything is torn down cleanly — no leaked browser processes.

session.goto(url) navigates, then waits for `networkidle`. Your app is a SPA: it loads, *then* fires API calls to fetch doctors, etc. The `networkidle` wait gives those calls time to finish before we read the page.

STAGE 1 Watch for errors — the Observer

src/agent/observer.py

This is the most subtle stage. The moment you create the `Observer`, its `__init__` attaches three **passive listeners** to the page:

```

page.on("console", self._on_console)    # JS console.error() messages
page.on("pageerror", self._on_pageerror) # uncaught JS exceptions
page.on("response", self._on_response)   # HTTP responses (catches 4xx / 5xx)

```

"Passive" means you never call these — they fire on their own, automatically, the entire time the browser is open. Every error the app logs, every failed network request, gets quietly appended to a list in the background. The Observer is a security camera you switch on at the start.

Later, when you want the footage, you call:

- **collect_errors()** — drains those lists and converts each raw string into a structured `Finding` (JS errors → high, network errors → medium, console errors → low), then clears the lists.
- **check_page()** — an *active* check. It runs a JS query right now: how many elements have `aria-invalid="true"`, class `.error`, `role="alert"`, or match `:invalid`? If > 0, it emits one `Finding`.

This is the function that caught your validation error. After you clicked submit, the form marked one field invalid, and `check_page()` counted it. That `FINDINGS` block you saw is the Observer doing exactly its job.

STAGE 2

Read the page — the Snapshotter

src/browser/snapshotter.py — the heart of the system

Its job: turn a messy live web page into a clean Python list of `Element` objects.

Step 2a — grab every interactive thing

```
handles = await page.query_selector_all(
    "input, button, a, select, textarea, "
    "[role='button'][aria-haspopup='listbox'], "
    "[role='combobox']"
)
```

The first five are normal HTML. The last two are the **MUI Select traps** — Material UI doesn't use real `<select>` tags, it uses styled `<div>`s that pretend to be dropdowns via `role="combobox"`. Without those, we'd be blind to your Doctor and Visit Reason dropdowns.

Step 2b — for each handle, build one Element

The loop runs once per handle. Tracing **the Doctor dropdown**:

```
if await handle.get_attribute("aria-hidden") == "true":
    continue # skip MUI's invisible phantom helpers

tag          = "div"
role         = "combobox"
widget_type  = "mui_select" # because role == "combobox"
```

That single `widget_type` flag is what later tells the executor: "don't use a normal fill, use the special dropdown dance." For First Name (a real `<input>`) it would be `"native"`.

Then the new part you added:

```
options = await self._extract_options(handle, tag, widget_type)
```

`_extract_options` is what stops the LLM from hallucinating. For an MUI Select it opens the popup, reads every option text → `['Dr. Vivek BHARAMBE', 'Dr. Sujata BHARAMBE', 'Other']`, presses Escape, and waits for the backdrop to clear. (That's the brief flash during snapshot.) For a real `<select>` it reads the `<option>` children directly. For a text input it returns `None`. Now the Element *carries its real menu with it*.

Three helper functions deserve unpacking — they caused half your past bugs:

`_resolve_name` — "what's this field's human label?"

Tries clues in order, returns the first that works:

1. **MUI walk-up:** find the closest `.MuiFormControl-root`, grab its label. This is what gives "Doctor" instead of the current value "Dr. Vivek BHARAMBE" (fixed your "Today" vs "Time" bug).
2. `aria-label` attribute
3. `aria-labelledby` (points to another element's id)
4. `<label for="...">` tied to the input's id
5. `placeholder` text
6. for buttons/links: the visible inner text
7. give up → `"(unnamed)"`

This ladder exists because every web app labels fields differently. The agent has to be a detective.

`_infer_kind` — "what type of data does this want?"

Returns one of `email / phone / date / name / address / currency / search / password / unknown` by checking the HTML `type`, the `autocomplete` attribute, then keyword-matching the label/id/placeholder. Your "Phone" field's label contains "phone" → `kind = "phone"`, which later tells the LLM to generate a phone number.

`_pick_selector` — "how do I find this element again later?"

The snapshot happens now, but we fill the form later, so we need a reliable address. It builds candidate selectors strongest-first (`data-testid` → clean `#id` → `[name=...]` → text) and returns the first that matches exactly one element. If nothing unique works, it falls back to a positional path.

This is why your fill selectors look like that ugly `div > div > main > ...` mess. Your MUI inputs have autogenerated React ids (like `:r5l:`) which `_looks_autogenerated` rejects (they contain `:`, which breaks CSS), and no `name` / `data-testid` — so it falls all the way back to the positional path. Ugly, but unique, so it works.

The result

```
Element(  
  tag="div",  
  name="Doctor",  
  selector="div > div > main > ... > div",  
  widget_type="mui_select",  
  in_form=True,  
  options=["Dr. Vivek BHARAMBE", "Dr. Sujata BHARAMBE", "Other"],  
)
```

`extract_elements` returns a **list of these**, one per field plus the buttons. That list is the agent's entire understanding of the page. Everything downstream works off this list — nobody touches the live page again until it's time to act.

STAGE 3 Understand the app — infer_context

src/agent/context.py

build_context_blob makes a compact text description of the page (title + headings + first 500 chars of text, plus a sample of up to 25 visible elements). It describes the page to the AI.

infer_context sends that blob to the LLM: "tell me the language, country, currency, domain (healthcare? e-commerce?), is the user logged in, what app type." The LLM replies as a structured `InferredContext` object.

For your waitlist page it should return something like `language="en", country_hint="IN", domain="healthcare"`. This is the stage meant to tell the data generator "this is an Indian hospital." If the phone field got a US number, this stage likely guessed `country_hint` wrong (or left it null) — a thread to pull on later.

The point: the same field means different things in different apps. A "name" field in a US shop wants "John Smith"; in your hospital it might want an Indian name. Context is how the agent adapts.

STAGE 4 Invent the data — DataGenerator.happy_path

src/data/generator.py — produces a dict of {selector → value to type}

Step 4a — fillable(elements): which fields do we fill?

Keeps only things you can put data into: `textarea`, `select`, anything with `widget_type == "mui_select"`, and text-like `input`s (skipping submit/button/hidden/file). Buttons and links get dropped here. For the waitlist that leaves seven fillable fields.

Step 4b — build field_lines

```
f"{e.selector} | kind={e.semantic_kind} | label={e.name}"
+ (f" | options={e.options}" if e.options else "")
```

One line per field. For Doctor (with your new options change):

```
... | kind=unknown | label=Doctor | options=['Dr. Vivek BHARAMBE', 'Dr. Sujata BHARAMBE',
```

For First Name (no options): `... | kind=name | label=First Name`. See the difference? Doctor now hands the LLM its menu. First Name doesn't have one, so the LLM is free to invent "Emily."

Step 4c — the prompt + the LLM call

The prompt says "generate realistic test data... and **IMPORTANT: if a field has options=[...], pick EXACTLY from that list.**" The LLM returns a `FormFill` object — just `{values: dict[str, str]}`:

```
{
  "...First Name input": "Emily",
  "...Phone input": "+1 650-123-4567",    # still wrong – no options to constrain it
  "...Doctor div": "Dr. Vivek BHARAMBE", # NOW correct – picked from options
}
```

This is the exact stage your last fix targeted. Before, Doctor had no options, so the LLM guessed "Dr. Rachel Kim." Now it must pick from the real list. Phone and Medical Record still have no options, so they're still free-form — that's the next problem to solve.

The returned dict is **keyed by selector** — the same selector strings the snapshotter generated. That's the join key between "what to type" and "where to type it."

STAGE 5 Type it in — Executor.fill_form

src/agent/executor.py

Step 5a — fill_form: rejoin values to elements

```
by_sel = {e.selector: e for e in elements}    # selector → full Element
for selector, value in values.items():
    results.append(await self._fill_one(by_sel.get(selector), selector, value))
```

The LLM gave us `{selector: value}`, but to fill correctly we need to know what *kind* of widget each selector is. So we rebuild a lookup from selector → full Element (which knows `widget_type`, `tag`).

Step 5b — _fill_one: the dispatcher

```
if widget_type == "mui_select" → self._mui_select(...)    # Doctor, Visit Reason
elif tag == "select"          → self._select(...)        # real <select>
elif input checkbox/radio    → self.page.check(...)
elif date field              → fill + press Escape
else                          → self.page.fill(...)        # plain text (First Name)
```

Every branch is wrapped in try/except. Success → `ActionResult(ok=True)` ; failure → `ok=False` with the error. **That's your FILLS log** — one ActionResult per field. This is why dispatching on `widget_type` matters: First Name uses `page.fill`, but Doctor is a `<div>` — `page.fill` would explode on it.

Step 5c — _mui_select: the dropdown dance

1. click the trigger → popup opens
2. wait for `[role='listbox']` to appear
3. read all options into `candidates`
4. exact match: is the LLM's value a non-placeholder exact match? → pick it
5. else → fall back to first real non-placeholder option
6. click that option
7. wait for listbox to vanish **and** the MUI backdrop to clear (your backdrop fix — otherwise the next click hits an invisible overlay)

The `_is_placeholder` helper skips rows like "Select Doctor" so we never pick the do-nothing placeholder. The `print` lines are your diagnostics — the `[mui_select] picking idx=0 ...` output.

STAGE 6 Click submit, safely

find_submit + Executor.click + SafetyGate

find_submit — pick the right button

The function you just upgraded. Priority order:

1. `<button type="submit">` inside a `<form>`
2. in-form button whose label matches submit words (apply, save, add, ...)
3. **last** button inside the form (submits sit at the bottom)
4. word-matched button anywhere
5. give up (return None) — refuse to click a random header button

"Add to Waiting List" got picked because it's in the form. **This is the fix that stopped it clicking the avatar.** The `in_form` flag (set in the snapshotter via `el.closest('form')`) powers rules 1–3.

Executor.click — but first, the safety check

```
verdict = await self.gate.evaluate(element)
if verdict.risk == "destructive":
    return ActionResult(action="skip", ...) # refuse to click
```

Before any click, the **SafetyGate** evaluates the button: (1) a hard danger word like "delete account" → block immediately; (2) an ambiguous word → ask the cheap LLM "safe/destructive/uncertain?"; (3) otherwise → safe. "Add to Waiting List" is safe, so the click proceeds. This guardrail stops the autonomous agent from clicking "Delete Patient" or "Charge Card" while exploring. The agent is loose in a real app — this is its conscience.

STAGE 7 Collect the verdict

```
findings = observer.collect_errors() + await observer.check_page()
```

Back to the Observer from Stage 1. Now we cash in: `collect_errors()` drains everything the passive camera recorded (JS crashes, failed network calls), and `check_page()` does a fresh DOM scan for validation markers right now. On your run, `collect_errors()` was empty but `check_page()` found 1 invalid field → the single FINDINGS entry. **That's the agent reporting a bug it found — the entire point of the project.**

The whole flow at a glance



The two honest weak spots right now

1. **Text fields are still LLM-blind.** Doctor/Visit Reason are now constrained by `options`, but Phone, Age, Medical Record have no constraints — the LLM guesses formats and locale. That's almost certainly what caused your one validation error. Fix = feed it locale + HTML `pattern` / `maxlength` / `placeholder` hints, just like we did with options.
2. **The Observer reports *that* a field is invalid, not *which* one.** `check_page()` just counts. To make findings actionable, it should read the error message text next to the invalid input.

Both are natural next steps — and you now understand the machine well enough that either one will make sense when we tackle it.